

What this template gives you

A new project that is secure, tested, and reviewed by default. It works for Python, JavaScript/TypeScript, Go, Rust, and Java - it detects your language and runs the right tools automatically.

You get	What it means for you
Secret protection	You cannot accidentally commit passwords or API keys.
Automatic checks	Lint, tests, and security scans/vulnerabilities scans run on every pull request.
Controlled releases	Code flows feature - dev - staging - main; nothing skips a stage.
Required reviews	The right people must approve before code reaches production.

First-time setup (about 10 minutes, once per new project)

1. **Create your repository** - on GitHub, click "Use this template".
2. **Install locally** - in your project folder, run:

```
make install
```

This sets up dependencies and turns on the secret-protection hooks.

3. **Pick your reviewers** - open .github/CODEOWNERS and replace @your-tech-lead and @your-project-owner with real GitHub usernames (keep the @). These people must have access to the repo.
4. **Ignore your language's files** - open .gitignore and uncomment the section for your stack (Python, Node, Go, etc.).
5. **Turn on branch protection** - this makes the reviews and checks required. With the GitHub CLI installed and admin access:

```
bash scripts/setup-branch-protection.sh OWNER/REPO
```

6. **For longer projects, enable full checks** - in GitHub: **Settings** - **Secrets and variables** - **Actions - Variables**, add CI_ENABLED = true.

You are now ready to work.

The commands you will use

Run these from your project folder. They work the same locally and in CI.

Command	Use it to
make install	Set up the project (run once).
make format	Auto-format your code.
make lint	Check code style and quality.
make test	Run the tests.
make coverage	Run tests and check coverage.

The everyday flow

1. Create a branch from dev, named feature/your-change.
2. Make your change. Run make lint and make test before committing.
3. Commit and push. (Secrets are checked automatically.)
4. Open a pull request **into dev**. Fill in the description template.
5. Wait for the checks to pass and get your review approval.

6. Once mature, promote dev - staging - main through pull requests.

The golden rule: code only moves one direction and never skips a stage. A pull request that jumps a stage (for example dev - main) is blocked.

How to write a pull request

Give every pull request a clear title and a real description - and template is provided

Title - start with the type of change:

feat: add password reset (a new feature)
fix: correct totals on invoices (a bug fix)
docs: update setup guide (documentation)

Description - answer four things:

- What changed and why (in plain language).
- The task or issue it relates to.
- How you tested it.
- Whether it touches anything sensitive (login, payments, data).

If something is blocked

You see	What to do
A red check on your PR	Open it, read the message, fix the issue, push again.
"may only receive from..."	You targeted the wrong branch. Open the PR against the correct stage.
A secret was detected	Remove it, use an environment variable instead, and rotate the secret.
Waiting on review	Make sure your reviewers are set in CODEOWNERS and have repo access.

Need more detail on any step? See README.md - it is the full reference.